

SECURITY MONITOR

Sistema de Monitoramento de Equipamentos

Versão 2.1 - Multi-Área

DOCUMENTAÇÃO TÉCNICA

Autor: Erik Thiago Barros Pinheiro

Departamento: TI Segurança Patrimonial

Empresa: Hidrovias do Brasil

Data: 14/02/2026

Documento confidencial - Uso interno Hidrovias do Brasil

ÍNDICE

1. Visão Geral do Sistema	
2. Arquitetura e Tecnologias	
3. Estrutura de Arquivos	
4. Banco de Dados	
5. Models (ORM)	
6. Rotas e Blueprints	
7. API REST - Endpoints	
8. Sistema de Autenticação e Permissões	
9. Serviço de Ping (Monitoramento)	
10. Sistema Multi-Área e Agentes Remotos	
11. Serviço de BI e Exportação	
12. Frontend e Interface	
13. Configuração e Deploy	
14. Fluxos Operacionais	

1. Visão Geral do Sistema

O **Security Monitor v2.1** é um sistema web desenvolvido para a **Hidrovias do Brasil** com o objetivo de monitorar em tempo real o status de equipamentos de segurança patrimonial, incluindo câmeras de CFTV e controles de acesso distribuídos em múltiplas áreas geográficas.

O sistema realiza pings automáticos periódicos (a cada 5 segundos) em todos os equipamentos cadastrados, registrando dados de uptime, downtime e latência. Toda a informação é consolidada em dashboards, relatórios e exportações para Power BI.

1.1 Objetivos Principais

- Monitoramento 24/7 de câmeras e controles de acesso
- Detecção automática de falhas (equipamento OFFLINE)
- Cálculo de uptime/downtime mensal com meta de 95%
- Workflow de manutenção (relatório inicial/final)
- Exportação de dados para Power BI (CSV por área)
- Suporte multi-área com agentes remotos independentes
- Sistema de permissões granular por tela/ação

1.2 Tipos de Equipamento

Tipo	Contexto	Descrição
Camera	RFB	Câmeras de CFTV da Receita Federal do Brasil
Camera	Normais	Câmeras de CFTV convencionais da empresa
Controle	RFB	Controles de acesso da Receita Federal
Controle	Normais	Controles de acesso convencionais

1.3 Estados de Equipamento

Status	Cor	Descrição	Impacto no Ping
ONLINE	Verde	Equipamento respondendo ao ping	Pingado normalmente
OFFLINE	Vermelho	Sem resposta ao ping	Pingado normalmente
MANUTENCAO	Amarelo	Em manutenção (relatório inicial criado)	NAO e pingado
DESLIGADO	Cinza	Desligamento aprovado por admin	NAO e pingado

NOTA: Equipamentos com status MANUTENCAO ou DESLIGADO são excluídos do ciclo de ping automático. O tempo nestes estados NAO conta como downtime.

2. Arquitetura e Tecnologias

O Security Monitor utiliza uma arquitetura web MVC (Model-View-Controller) baseada no framework Flask, com comunicacao via API REST entre frontend e backend. O sistema suporta multiplas areas geograficas, cada uma podendo ter seu proprio agente remoto.

2.1 Stack Tecnologico

Camada	Tecnologia	Versao	Finalidade
Backend	Python + Flask	3.x / 3.0+	Servidor web e API REST
ORM	SQLAlchemy	2.0+	Mapeamento objeto-relacional
Banco de Dados	SQL Server	Express	Armazenamento persistente
Driver DB	pyodbc	5.0+	Conexao ODBC com SQL Server
Autenticacao	Flask-Login	0.6+	Gerenciamento de sessoes
Seguranca	Werkzeug (scrypt)	3.0+	Hash de senhas
Ping	ping3 / subprocess	4.0+	Verificacao de conectividade
Frontend	HTML/CSS/JS	-	Interface SPA-like
Graficos	Chart.js	4.4.1	Graficos interativos
Config	python-dotenv	1.0+	Variaveis de ambiente
Scheduler	APScheduler	3.10+	Tarefas agendadas
HTTP	requests	2.31+	Requisicoes HTTP (snapshots)

2.2 Diagrama de Arquitetura



2.3 Padroes de Projeto Utilizados

- **Application Factory** - create_app() e create_app_with_ping()
- **Blueprint** - Separacao modular de rotas (main_bp, auth_bp, api_bp, api_agente)
- **Context Manager** - DatabaseSession() para sessoes DB seguras
- **Decorator Pattern** - @permissao_requerida, @admin_requerido
- **Service Layer** - PingService, BIExportService
- **Thread Pool** - ThreadPoolExecutor para pings paralelos (50 workers)
- **Singleton** - ping_service = PingService() instancia global

3. Estrutura de Arquivos

```
SecurityMonitor/
+-- run.py                # Entry point (argparse: host, port, debug)
+-- SecurityMonitor.pyw   # Launcher Windows (sem console)
+-- INICIAR_SERVIDOR.bat  # Script de inicializacao Windows
+-- requirements.txt      # Dependencias Python
+-- s.py                 # Utilitario de teste de conexao DB
+-- .env                 # Variaveis de ambiente
|
+-- config/
|   +-- settings.py      # Classes Config (Dev/Prod/Test)
|   |
+-- app/
|   +-- __init__.py      # Application Factory + Blueprints
|   +-- database.py      # Engine SQLAlchemy + DatabaseSession
|   +-- decorators.py    # @permissao_requerida, @admin_requerido
|   |
|   +-- models/          # ORM Models (SQLAlchemy 2.0)
|   |   +-- __init__.py  # Exports de todos os models
|   |   +-- usuario.py   # Usuarios e autenticacao (script)
|   |   +-- equipamento.py # Equipamentos (25+ colunas)
|   |   +-- area.py      # Areas + AgenteHeartbeat
|   |   +-- permissao.py  # Permissoes por tela
|   |   +-- status_log.py # Log de mudancas de status
|   |   +-- ping_log.py   # Log de pings individuais
|   |   +-- relatorio.py  # Relatorios de manutencao
|   |   +-- solicitacao_deslig.py # Solicitacoes de desligamento
|   |   +-- configuracao_sistema.py # Configuracoes key-value
|   |   +-- snapshot_bi.py # Snapshots diarios/mensais (BI)
|   |
|   +-- routes/          # Blueprints de rotas
|   |   +-- main.py      # Rotas de paginas (233 linhas)
|   |   +-- auth.py      # Login, logout, alterar senha (127 lin)
|   |   +-- api.py       # API REST principal (~2900 linhas)
|   |   +-- api_agente.py # API p/ agentes remotos (~1500 linhas)
|   |
|   +-- services/        # Servicos de background
|   |   +-- ping_service.py # Servico de ping automatico (332 lin)
|   |   +-- bi_export_service.py # Exportacao CSV para Power BI (787 lin)
|   |   +-- assistente_service.py # Assistente de consultas
|   |
|   +-- templates/       # Templates HTML (Jinja2)
|   |   +-- base.html    # Template base (sidebar, navbar)
|   |   +-- login.html   # Tela de login
|   |   +-- dashboard.html # Dashboard principal
|   |   +-- cameras.html  # Listagem de equipamentos
|   |   +-- historico.html # Historico de uptime
|   |   +-- relatorios.html # Workflow de relatorios
|   |   +-- configuracoes.html # Configuracoes do sistema
|   |   +-- cadastro.html # Cadastro de equipamentos
|   |   +-- areas.html    # Gerenciamento de areas (admin)
|   |
|   +-- static/
|   |   +-- css/style.css # Estilos (~3500 linhas)
|   |   +-- js/app.js     # JavaScript principal (589 lin)
|   |   +-- js/assistente-ia.js # JavaScript do assistente
|   |
+-- database/            # Scripts SQL
|   +-- 01_CRIAR_BANCO_E_TABELAS.sql # Schema completo (~400 lin)
|
+-- exports/             # CSVs gerados para Power BI
|   +-- {AREA_CODE}/     # Uma pasta por area
|   +-- snapshots_diarios.csv
|   +-- snapshots_mensais.csv
```

4. Banco de Dados

O sistema utiliza **Microsoft SQL Server Express** com **Windows Authentication** (Trusted Connection). A conexao e gerenciada pelo SQLAlchemy com pool de conexoes otimizado (definido em `app/database.py`).

4.1 Configuracao de Conexao

```
# app/database.py
DATABASE_CONFIG = {
    'server': os.getenv('DB_SERVER', 'localhost\SQLEXPRESS'),
    'database': os.getenv('DB_NAME', 'SecurityMonitor'),
    'driver': os.getenv('DB_DRIVER', 'ODBC Driver 17 for SQL Server'),
    'trusted_connection': os.getenv('DB_TRUSTED', 'yes'),
}

# Connection String resultante:
mssql+pyodbc:///?odbc_connect=DRIVER={ODBC Driver 17};
SERVER=localhost\SQLEXPRESS;DATABASE=SecurityMonitor;
Trusted_Connection=yes
```

4.2 Pool de Conexoes (QueuePool)

Parametro	Valor	Descricao
pool_size	20	Conexoes permanentes no pool
max_overflow	40	Conexoes extras sob demanda (total max: 60)
pool_recycle	1800s	Reciclar conexoes a cada 30 minutos
pool_pre_ping	True	Verificar conexao antes de usar
pool_timeout	60s	Timeout para obter conexao do pool
echo	False	True para debug de SQL (desenvolvimento)

4.3 DatabaseSession - Context Manager

```
# Uso simplificado em todas as rotas e servicos:
class DatabaseSession:
    def __init__(self):
        self.db = SessionLocal()
    def __enter__(self):
        return self.db
    def __exit__(self, exc_type, exc_val, exc_tb):
        if exc_type:
            self.db.rollback()
        self.db.close()

# Exemplo de uso:
with DatabaseSession() as db:
    equipamentos = db.query(Equipamento).all()
```

4.4 Tabelas do Banco

Tabela	PK	Descricao	Indices
usuarios	INT IDENTITY	Usuarios do sistema	username+ativo

Tabela	PK	Descricao	Indices
permissoes	INT IDENTITY	Permissoes por tela	usuario_id, tela
equipamentos	INT IDENTITY	Cameras e controles	tipo+contexto, status
areas	INT IDENTITY	Areas monitoradas	codigo (UNIQUE)
status_log	BIGINT IDENTITY	Historico de status	equipamento+data
ping_log	BIGINT IDENTITY	Log de cada ping	equipamento+data
relatorios	INT IDENTITY	Relatorios manutencao	equipamento, tipo
solicit_desligamento	INT IDENTITY	Workflow desligamento	status, equipamento
config_sistema	INT IDENTITY	Config key-value	chave (UNIQUE)
agentes_heartbeat	BIGINT IDENTITY	Heartbeat agentes	area_id+timestamp
snapshots_diarios	BIGINT IDENTITY	Snapshot diario (BI)	equip+data
snapshots_mensais	BIGINT IDENTITY	Snapshot mensal (BI)	equip+mes

4.5 Tabela: equipamentos (detalhada)

Coluna	Tipo SQL	Descricao
id	INT IDENTITY PK	ID auto-incremento
tipo	VARCHAR(20) NOT NULL	camera ou controle (CHECK)
nome	VARCHAR(100) NOT NULL	Nome do equipamento
descricao	VARCHAR(255) NOT NULL	Descricao detalhada
ip	VARCHAR(15) UNIQUE	Endereco IP
fabricante	VARCHAR(50)	Fabricante (Axis, Hikvision, etc.)
modelo	VARCHAR(50)	Modelo do equipamento
contexto	VARCHAR(20) NOT NULL	rfb ou normais (CHECK)
status_atual	VARCHAR(20) DEFAULT 'OFFLINE'	ONLINE/OFFLINE/MANUTENCAO/DESLIGADO
ultimo_ping	DATETIME	Timestamp do ultimo ping
ping_ms	INT	Latencia do ultimo ping (ms)
uptime_total_segundos	BIGINT DEFAULT 0	Acumulador total de uptime
downtime_total_segundos	BIGINT DEFAULT 0	Acumulador total de downtime
uptime_mensal_segundos	BIGINT DEFAULT 0	Uptime do mes corrente
downtime_mensal_segundos	BIGINT DEFAULT 0	Downtime do mes corrente
mes_ano_referencia	VARCHAR(7)	Mes de ref. (YYYY-MM)
ultima_mudanca_status	DATETIME	Quando mudou status
area_id	INT FK NOT NULL	Area a que pertence
camera_usuario	VARCHAR(50)	Login HTTP/RTSP da camera
camera_senha	VARCHAR(100)	Senha HTTP/RTSP da camera

Coluna	Tipo SQL	Descricao
camera_porta	INT DEFAULT 80	Porta HTTP da camera
camera_porta_rtsp	INT DEFAULT 554	Porta RTSP da camera
ativo	BIT DEFAULT 1	Se equipamento esta ativo

5. Models (ORM)

Os models utilizam SQLAlchemy 2.0 com mapeamento declarativo. Base declarativa definida em **app/database.py**. Todos estao no diretorio **app/models/**.

5.1 Lista de Models

Model	Tabela	Arquivo	Responsabilidade
Usuario	usuarios	usuario.py	Autenticacao, senha script, UserMixin
Permissao	permissoes	permissao.py	Permissao por tela (pode_ver/pode_editar)
Equipamento	equipamentos	equipamento.py	Cameras e controles (25+ colunas)
Area	areas	area.py	Areas geograficas + config agente
AgenteHeartbeat	agentes_heartbeat	area.py	Log de heartbeat do agente remoto
StatusLog	status_log	status_log.py	Historico de mudancas de status
PingLog	ping_log	ping_log.py	Log individual de cada ping
Relatorio	relatorios	relatorio.py	Relatorios de manutencao (inicial/final)
SolicitacaoDeslig.	solicit_desligamento	solicitacao_*.py	Workflow desligamento
ConfiguracaoSistema	config_sistema	configuracao_*.py	Configuracoes key-value
SnapshotDiario	snapshots_diarios	snapshot_bi.py	Snapshot diario para BI
SnapshotMensal	snapshots_mensais	snapshot_bi.py	Snapshot mensal para BI

5.2 Model: Usuario (usuario.py)

Herda de **UserMixin** (Flask-Login) e **Base** (SQLAlchemy). Implementa autenticacao com hash **script** via Werkzeug:

```
class Usuario(Base, UserMixin):
    __tablename__ = 'usuarios'

    # Campos: id, username, password_hash, nome_completo,
    #          email, is_admin, ativo, ultimo_acesso, criado_em

    def set_password(senha, validar_forca=True):
        # generate_password_hash(password, method='script')

    def check_password(senha):
        # check_password_hash(self.password_hash, password)

    def validar_forca_senha(senha): # staticmethod
        # 12+ chars, maiusc, minusc, digito, especial

    def tem_permissao(tela, tipo='ver'):
        # Admin = acesso total, senao busca Permissao no DB
```

5.3 Model: Equipamento (equipamento.py)

Model principal com 25+ colunas. Constantes e metodos:

- **STATUS_ONLINE/OFFLINE/MANUTENCAO/DESLIGADO** - constantes de status
- **TIPO_CAMERA/TIPO_CONTROLE** - constantes de tipo

- **CONTEXTO_RFB/CONTEXTO_NORMAIS** - constantes de contexto
- **format_uptime()** - Formata uptime_total_segundos para leitura humana (Xd Xh Xm)
- **format_downtime()** - Formata downtime_total_segundos
- **format_uptime_mensal()** - Formata contadores mensais
- **format_tempo_offline_atual()** - Tempo offline desde ultima mudanca
- **to_dict()** - Serializacao completa para JSON (API REST)

CONTAGEM INCREMENTAL: Os contadores uptime_mensal e downtime_mensal sao incrementados a cada ciclo de ping (5s) com base no tempo desde o ultimo ping. O campo mes_ano_referencia garante que os contadores so somam no mes correto.

5.4 Model: Area (area.py)

Representa uma area geografica com configuracao de agente remoto:

- **codigo** - Codigo unico (ex: ITA, SNT, PAR)
- **agente_ativo** - Flag se agente esta habilitado
- **agente_url** - URL do agente remoto
- **agente_api_key** - Chave de autenticacao do agente
- **agente_status** - ATIVO, INATIVO, ERRO, OFFLINE
- **agente_ultimo_heartbeat** - Timestamp do ultimo heartbeat

6. Rotas e Blueprints

O sistema utiliza 4 Blueprints Flask registrados em `app/__init__.py` pela Application Factory:

```
# app/__init__.py - create_app()
app.register_blueprint(main_bp)           # /
app.register_blueprint(auth_bp, url_prefix='/auth') # /auth
app.register_blueprint(api_bp, url_prefix='/api') # /api
app.register_blueprint(api_agente)        # /api
```

Blueprint	Prefixo	Arquivo	Responsabilidade
main_bp	/	routes/main.py	Paginas HTML (dashboard, cameras, etc.)
auth_bp	/auth	routes/auth.py	Autenticacao (login/logout/alterar-senha)
api_bp	/api	routes/api.py	API REST principal (~2900 linhas)
api_agente	/api	routes/api_agente.py	API agentes remotos (~1500 linhas)

6.1 Rotas de Pagina (main_bp)

Rota	Metodo	Permissao	Descricao
/	GET	login	Redirect para dashboard
/dashboard	GET	dashboard:ver	Dashboard principal
/cameras/rfb	GET	cameras_rfb:ver	Cameras RFB
/cameras/normais	GET	cameras_normais:ver	Cameras normais
/controles/rfb	GET	controles_rfb:ver	Controles RFB
/controles/normais	GET	controles_normais:ver	Controles normais
/historico	GET	historico:ver	Historico de uptime
/relatorios	GET	relatorios:ver	Workflow de relatorios
/configuracoes	GET	configuracoes:ver	Configuracoes do sistema
/cadastro	GET	cadastro:ver	Cadastro de equipamentos
/areas	GET	admin_requerido	Gerenciamento de areas

6.2 Rotas de Autenticacao (auth_bp)

Rota	Metodo	Descricao
/auth/login	GET, POST	Formulario de login + autenticacao
/auth/logout	GET	Encerra sessao (logout_user)
/auth/alterar-senha	GET, POST	Alteracao de senha propria

FILTRO POR AREA: Todas as rotas de main.py leem o cookie 'areaAtual' via `get_area_id_from_cookie()`. A API usa `?area_id=X` como query param. Cada operador ve apenas equipamentos da area selecionada.

7. API REST - Endpoints

A API REST (~2900 linhas em **routes/api.py**) processa todas as operacoes de CRUD e dados em tempo real. Padrao de resposta:

```
{ "success": true|false, "data": { ... }, "error": "msg", "area_id": 1 }
```

7.1 Equipamentos (CRUD)

Endpoint	Metodo	Descricao
/api/equipamentos	GET	Listar (filtros: tipo, contexto, status, busca, area_id)
/api/equipamentos/<id>	GET	Obter detalhes de um equipamento
/api/equipamentos	POST	Criar novo equipamento (area_id obrigatorio)
/api/equipamentos/<id>	PUT	Atualizar (inclui credenciais camera)
/api/equipamentos/<id>	DELETE	Excluir equipamento
/api/equipamentos/distribuicao	GET	Distribuicao por tipo/contexto
/api/equipamentos/pendentes	GET	Equipamentos OFFLINE aguardando relatorio
/api/equipamentos/manutencao	GET	Equipamentos em MANUTENCAO
/api/equipamentos/desligados	GET	Equipamentos DESLIGADOS

7.2 Ping e Dashboard

Endpoint	Metodo	Descricao
/api/equipamentos/<id>/ping	POST	Executa ping manual e atualiza status
/api/ping	POST	Ping em IP especifico (teste de conectividade)
/api/equipamentos/<id>/snapshot	GET	Captura imagem da camera (HTTP Digest)
/api/dashboard/stats	GET	Contadores agrupados por status
/api/dashboard/uptime-historico	GET	Historico uptime (30 dias)
/api/dashboard/ultimas-mudancas	GET	Ultimas 20 mudancas de status

7.3 Relatorios e Solicitacoes

Endpoint	Metodo	Descricao
/api/relatorios	GET	Listar relatorios (filtros: tipo, equipamento)
/api/relatorios	POST	Criar relatorio inicial ou final
/api/solicitacoes	GET	Listar solicitacoes de desligamento
/api/solicitacoes	POST	Criar solicitacao de desligamento
/api/solicitacoes/<id>/aprovar	POST	Aprovar solicitacao = DESLIGADO
/api/solicitacoes/<id>/rejeitar	POST	Rejeitar solicitacao

7.4 Usuarios e Config

Endpoint	Metodo	Descricao
/api/usuarios	GET	Listar todos os usuarios
/api/usuarios	POST	Criar novo usuario (validacao senha forte)
/api/usuarios/<id>	PUT	Atualizar usuario e permissoes
/api/usuarios/<id>	DELETE	Excluir (nao pode excluir a si mesmo)
/api/configuracoes	GET	Listar configuracoes do sistema
/api/configuracoes	PUT	Atualizar configuracoes
/api/historico-detalhado/<id>	GET	Historico detalhado com graficos

8. Sistema de Autenticação e Permissões

8.1 Autenticacao (routes/auth.py)

Flask-Login para sessoes + Werkzeug (bcrypt) para hash de senhas:

- Usuario submete username + password via POST /auth/login
- Backend busca usuario ativo: Usuario.username + Usuario.ativo == True
- Senha verificada com check_password_hash() (bcrypt)
- Flask-Login cria sessao com login_user(usuario, remember=True/False)
- Campo ultimo_acesso atualizado com datetime.now()
- Redirect para next_page ou dashboard

8.2 Validacao de Senha Forte

Ao criar ou alterar senha, Usuario.validar_forca_senha() exige:

- Minimo **12 caracteres**
- Pelo menos 1 letra **maiuscula** (A-Z)
- Pelo menos 1 letra **minusculta** (a-z)
- Pelo menos 1 **digito** numerico (0-9)
- Pelo menos 1 **caractere especial** (!@#\$%^&*()_+~[]{}|;:,.<>?)

EXCECAO: A rota /auth/alterar-senha valida apenas minimo de 4 caracteres (validacao simplificada). A validacao completa e aplicada em set_password() com validar_forca=True.

8.3 Permissoes por Tela (decorators.py)

```
# Decorator de permissao:
@permissao_requerida('configuracoes', 'editar')
def minha_rota():
    ...

# Decorator de admin:
@admin_requerido
def rota_admin():
    ...
```

Tela	pode_ver	pode_editar	Descricao
dashboard	Ver dashboard	Editar	Tela principal
cameras_rfb	Ver cameras RFB	Editar	Cameras Receita Federal
cameras_normais	Ver cameras normais	Editar	Cameras convencionais
controles_rfb	Ver controles RFB	Editar	Controles Receita Federal
controles_normais	Ver controles normais	Editar	Controles convencionais
historico	Ver historico	Editar	Dados historicos
relatorios	Ver relatorios	Criar	Workflow de manutencao

Tela	pode_ver	pode_editar	Descricao
configuracoes	Ver config	Editar	Usuarios e sistema
cadastro	Ver cadastro	Cadastrar	CRUD de equipamentos

8.4 Headers de Seguranca (app/__init__.py)

Adicionados via @app.after_request em add_security_headers():

Header	Valor	Protecao
X-Frame-Options	SAMEORIGIN	Previne clickjacking
X-Content-Type-Options	nosniff	Previne MIME sniffing
X-XSS-Protection	1; mode=block	Filtro XSS do navegador
Referrer-Policy	strict-origin-when-cross-origin	Controle de referrer
Permissions-Policy	geolocation=(), microphone=(), camera=()	Desabilita recursos

9. Serviço de Ping (Monitoramento)

O **PingService** (`app/services/ping_service.py`, 332 linhas) é o núcleo do monitoramento em tempo real. Roda como thread daemon e executa ciclos de ping utilizando `ThreadPoolExecutor` com 50 workers paralelos.

9.1 Configurações (constantes no módulo)

Constante	Valor	Descrição
INTERVALO_PING	5 segundos	Intervalo entre ciclos de ping
TIMEOUT_PING	2 segundos	Timeout individual por ping
MAX_WORKERS	50 threads	Threads paralelas no <code>ThreadPoolExecutor</code>
TIMEOUT_AGENTE	180 segundos	Tempo sem heartbeat para ativar fallback

9.2 Fluxo do Ciclo de Ping (`_executar_ciclo`)

```
INICIO DO CICLO
|
+- 1. Consultar areas com agente_api_key configurada
|   +- Agente ATIVO (heartbeat < 180s) -> NAO pinga
|   +- Agente OFFLINE (heartbeat > 180s) -> PINGA (fallback)
|   +- Area.agente_status = 'OFFLINE' se agente caiu
|
+- 2. Buscar equipamentos ativos (excluir MANUTENCAO/DESLIGADO)
|   +- Excluir equipamentos de areas com agente ATIVO
|
+- 3. Pings em paralelo (ThreadPoolExecutor, 50 workers)
|   +- Tenta ping3 (library) primeiro
|   +- Fallback: ping do SO (subprocess)
|
+- 4. Processar resultados por equipamento:
|   +- Atualizar ultimo_ping e ping_ms
|   +- duracao = (agora - ultimo_ping).seconds
|   +- duracao = min(duracao, 300) # Cap 5 minutos
|   +- Incrementar uptime_total / downtime_total
|   +- Incrementar uptime_mensal / downtime_mensal
|     (somente se mes_ano_referencia == mes atual)
|   +- Se mudou status: criar StatusLog
|
+- 5. db.commit()
|
+- 6. Atualizar self.stats (metricas do ciclo)
|
+- 7. time.sleep(5) -> Repetir
```

9.3 Método `_ping(ip)`

```
def _ping(self, ip):
    # 1. Tentar ping3 (library Python - mais rapido)
    from ping3 import ping as ping3_func
    resultado = ping3_func(ip, timeout=2)
    if resultado: return True, int(resultado * 1000) # ms

    # 2. Fallback: ping do sistema operacional
    subprocess.run(['ping', '-n', '1', '-w', '2000', ip])
    # Windows: -n 1 -w 2000 | Linux: -c 1 -W 2
```

9.4 Contagem Incremental de Uptime/Downtime

```
# Calculo baseado no tempo desde o ULTIMO PING
# (nao desde ultima_mudanca_status - evita duplicacao)
duracao_ciclo = (datetime.now() - equipamento.ultimo_ping).seconds
duracao_ciclo = min(duracao_ciclo, 300) # Cap em 5 minutos

# Incrementar contadores do status ANTERIOR:
if status_anterior == 'ONLINE':
    equipamento.uptime_total_segundos += duracao_ciclo
    if mes_atual_ok: # mes_ano_referencia == YYYY-MM atual
        equipamento.uptime_mensal_segundos += duracao_ciclo
elif status_anterior == 'OFFLINE':
    equipamento.downtime_total_segundos += duracao_ciclo
    if mes_atual_ok:
        equipamento.downtime_mensal_segundos += duracao_ciclo
```

IMPORTANTE: O reset dos contadores mensais e feito APENAS pelo BIEExportService.rotina_meia_noite(). O PingService apenas verifica se mes_ano_referencia == mes atual antes de incrementar.

10. Sistema Multi-Área e Agentes Remotos

A versão 2.1 introduziu **múltiplas áreas geográficas**, cada uma podendo ter um **agente remoto** instalado localmente para pingar equipamentos com menor latência e sincronizar com o servidor central.

10.1 Conceito de Área

- **código** único (ex: ITA, SNT, PAR) - campo UNIQUE no banco
- Equipamentos pertencem a uma área (FK area_id NOT NULL)
- Operadores filtram por área via cookie 'areaAtual'
- Admin pode gerenciar áreas na tela /areas

10.2 API do Agente (routes/api_agente.py)

Endpoint	Método	Descrição
/api/agente/heartbeat	POST	Registrar heartbeat (auth por API key)
/api/agente/equipamentos	GET	Buscar equipamentos da área
/api/agente/ping-results	POST	Enviar resultados de ping
/api/agente/ping-results-batch	POST	Enviar resultados em lote (buffer sync)
/api/agente/snapshots-diarios-batch	POST	Sincronizar snapshots diários
/api/agente/snapshots-mensais-batch	POST	Sincronizar snapshots mensais
/api/agente/status	GET	Status do agente e estatísticas
/api/agente/config	GET	Configuração remota do agente

10.3 Mecanismo de Fallback (PingService)

```
# ping_service.py - _executar_ciclo()

SERVIDOR CENTRAL
|
|-- Área COM agente + heartbeat RECENTE (< 180s)
|   |-- ids_agente_ativo[] -> NÃO pinga
|   |-- area.agente_status permanece atual
|   |
|-- Área COM agente + heartbeat ANTIGO (> 180s)
|   |-- ids_agente_fallback[] -> PINGA (servidor assume)
|   |-- area.agente_status = 'OFFLINE'
|   |-- Log: 'FALLBACK - agente offline'
|   |
|-- Área SEM agente (agente_api_key vazio)
|   |-- SEMPRE pinga (comportamento padrão)
```

AUTENTICACAO: Agentes autenticam via API key (campo Area.agente_api_key). O heartbeat registra: status, latencia_ms, equipamentos_pingados, equipamentos_online.

11. Serviço de BI e Exportação

O **BIExportService** (app/services/bi_export_service.py, 787 linhas) calcula metricas de uptime, gera snapshots diarios/mensais e exporta CSVs para consumo pelo **Power BI**. Roda em thread daemon separada iniciada pelo run.py.

11.1 Inicializacao (run.py)

```
# run.py - run_bi_scheduler_thread()
bi_thread = threading.Thread(
    target=run_bi_scheduler_thread,
    args=(args.bi_dir, args.bi_interval), # default: exports, 20
    daemon=True
)
bi_thread.start()
```

11.2 Rotinas Agendadas

Rotina	Frequencia	Descricao
rotina_20_minutos()	A cada 20 min (configuravel)	Atualiza snapshots diarios + exporta CSVs
rotina_meia_noite()	00:00-00:20 (automatica)	Finaliza dia, salva mensal, reseta contadores

11.3 Meta de Uptime

```
META_UPTIME = 95.0 # Constante da classe

# Calculo de percentual mensal:
dias_mes = calendar.monthrange(ano, mes)[1]
total_horas_mes = dias_mes * 24 * total_equipamentos
uptime_horas = uptime_segundos / 3600
uptime_percent = (uptime_horas / total_horas_mes) * 100

meta_atingida = (uptime_percent >= 95.0)
```

11.4 Exportacao CSV por Area

```
exports/
+-- ITA/
|   +-- snapshots_diarios.csv # 21 colunas
|   +-- snapshots_mensais.csv # 18 colunas
+-- SNT/
|   +-- snapshots_diarios.csv
|   +-- snapshots_mensais.csv
+-- ...
```

11.5 Headers dos CSVs

Diario: Data_Dia, Contexto, Mes_Ano, Total_Cameras, Cameras_Online_Dia, Cameras_Offline_Dia, Uptime_Segundos_Mensal, Downtime_Segundos_Mensal, Uptime_Percent_Mensal, Faltante_95_Percent_Mensal, Uptime_Percent_Anual, Incidentes_Ativos

Mensal: Data_Relatorio, Contexto, Mes_Ano, Total_Cameras, Uptime_Segundos_Mensal, Downtime_Segundos_Mensal, Uptime_Percent_Mensal, Faltante_95_Percent_Mensal, Uptime_Percent_Anual

VIRADA DE MES: rotina_meia_noite() executa: 1) Salva SnapshotMensal com dados acumulados, 2) Reseta contadores mensais (uptime_mensal = 0, downtime_mensal = 0), 3) Atualiza mes_ano_referencia para o novo mes. Nenhum dado e perdido.

12. Frontend e Interface

Frontend em HTML/CSS/JS puro com navegacao SPA-like via atributo **data-spa-link**. Templates renderizados pelo Jinja2 a partir de **app/templates/base.html**.

12.1 Telas Principais

Tela	Template	Descricao
Login	login.html	Autenticacao com username/senha
Dashboard	dashboard.html	Cards de status, graficos Chart.js 4.4.1
Cameras/Controles	cameras.html	Tabela com status, uptime, ping, acoes
Historico	historico.html	Graficos de uptime por periodo
Relatorios	relatorios.html	Workflow inicial/final, desligamento
Configuracoes	configuracoes.html	CRUD de usuarios, permissoes, config
Cadastro	cadastro.html	CRUD de equipamentos
Areas	areas.html	Gerenciamento de areas e agentes (admin)

12.2 JavaScript Principal (app.js - 589 linhas)

- **ThemeManager** - Alternancia dark/light mode via data-theme
- **SidebarManager** - Controle da sidebar (toggle, mobile)
- **SPA Navigation** - Links com data-spa-link interceptam click
- **Auto-refresh** - Dashboard e tabelas atualizam a cada 30 segundos

12.3 Funcionalidades da Interface

- **Ver Detalhes** - Modal com graficos Chart.js (uptime, quedas, timeline)
- **Ping manual** - Botao para testar conectividade em tempo real
- **Snapshot** - Captura imagem da camera via HTTP Digest/Basic
- **Seletor de area** - Dropdown no navbar para trocar entre areas
- **Dark/Light mode** - Tema claro e escuro persistente
- **Exportar CSV** - Exportacao da listagem de equipamentos
- **Sidebar responsiva** - Adapta para mobile automaticamente

12.4 CSS (style.css - ~3500 linhas)

Design system completo com variaveis CSS para temas. Suporte a dark mode via atributo data-theme no HTML. Cards de status com cores: verde (online), vermelho (offline), amarelo (manutencao), cinza (desligado).

13. Configuração e Deploy

13.1 Requisitos do Sistema

Componente	Requisito	Observacao
Python	3.10+	Recomendado: 3.11 ou 3.12
SQL Server	Express 2019+	Windows Authentication
ODBC Driver	Driver 17	Necessario no servidor
SO	Windows Server/10/11	Para Windows Auth
Rede	Acesso aos IPs	Portas ICMP, 80, 554

13.2 Dependencias (requirements.txt)

Pacote	Versao	Finalidade
Flask	>=3.0.0	Framework web
Flask-Login	>=0.6.3	Gerenciamento de sessoes
Flask-WTF	>=1.2.0	Formularios e CSRF
Flask-SocketIO	>=5.3.0	WebSocket (opcional)
pyodbc	>=5.0.0	Driver ODBC para SQL Server
SQLAlchemy	>=2.0.0	ORM
Werkzeug	>=3.0.0	Utilitarios HTTP + hash script
python-dotenv	>=1.0.0	Variaveis de ambiente (.env)
APScheduler	>=3.10.0	Agendamento de tarefas
requests	>=2.31.0	Requisicoes HTTP (snapshots)
ping3	>=4.0.0	Ping ICMP via Python
python-dateutil	>=2.8.0	Utilitarios de data

13.3 Instalacao Completa

```
# 1. Criar ambiente virtual
python -m venv venv
venv\Scripts\activate

# 2. Instalar dependencias
pip install -r requirements.txt

# 3. Configurar .env
copy .env.example .env
# Editar: SECRET_KEY, DB_SERVER, DB_NAME

# 4. Criar banco de dados
# Executar: database/01_CRIAR_BANCO_E_TABELAS.sql
# (cria tabelas, indices, usuario admin, configuracoes)

# 5. Iniciar o servidor
python run.py
```

13.4 Argumentos de Linha de Comando (run.py)

```
python run.py [opcoes]

--host HOST      Host (default: 0.0.0.0)
--port PORT      Porta (default: 5000)
--debug          Ativar modo debug
--no-monitor     Desativar servico de ping
--no-bi          Desativar scheduler de BI
--export-bi      Exportar CSVs uma vez e sair
--bi-dir DIR     Diretorio exports (default: exports)
--bi-interval MIN Intervalo export em minutos (default: 20)
```

13.5 Modos de Execucao

Comando	Ping	BI	Uso
python run.py	SIM	SIM	Producao (tudo ativo)
python run.py --no-monitor	NAO	SIM	Sem ping (debug)
python run.py --no-bi	SIM	NAO	Sem BI export
python run.py --export-bi	-	1x	Exporta CSV e sai
python run.py --debug	SIM	SIM	Modo debug Flask

13.6 Formas de Iniciar

- **INICIAR_SERVIDOR.bat** - Script Windows com deteccao automatica do Python
- **SecurityMonitor.pyw** - Launcher Windows sem console (CREATE_NO_WINDOW)
- **python run.py** - Linha de comando direta

13.7 Configuracoes do Sistema (banco)

Chave	Default	Descricao
ping_intervalo_segundos	5	Intervalo entre ciclos de ping
ping_timeout_ms	2000	Timeout de cada ping (ms)
ping_max_workers	50	Threads paralelas de ping
tempo_offline_relatorio	10800	Seg. offline p/ pedir relatorio (3h)
retencao_ping_log_dias	90	Dias para manter logs de ping
retencao_status_log_dias	365	Dias para manter logs de status
tema_padrao	light	Tema padrao da interface (light/dark)

13.8 Classes de Configuracao (config/settings.py)

```
class Config:          # Base - SECRET_KEY, DB_*, PING_*, TELEGRAM_*
class DevelopmentConfig # DEBUG=True
class ProductionConfig  # DEBUG=False
class TestingConfig     # DEBUG=True, TESTING=True

# Selecao por FLASK_ENV:
get_config() -> config[os.getenv('FLASK_ENV', 'development')]
```

14. Fluxos Operacionais

14.1 Fluxo de Manutencao

EQUIPAMENTO OFFLINE

- |
- + 1. Operador identifica offline no dashboard
- |
- + 2. Cria RELATORIO INICIAL:
 - | +- Descricao do problema + Categoria
 - | +- POST /api/relatorios (tipo='inicial')
 - | +- Status muda para -> MANUTENCAO
 - | | (Equipamento PARA de ser pingado)
 - |
- + 3. Tecnico realiza a manutencao fisica
- |
- + 4. Cria RELATORIO FINAL:
 - | +- Referencia o relatorio inicial (relatorio_inicial_id)
 - | +- Descreve solucao aplicada
 - | +- Status muda para -> OFFLINE
 - | | (Equipamento VOLTA a ser pingado)
 - |
- + 5. Ping automatico detecta resposta
 - | +- Status muda para -> ONLINE

14.2 Fluxo de Desligamento

EQUIPAMENTO ATIVO

- |
- + 1. Operador cria SOLICITACAO DE DESLIGAMENTO
 - | +- POST /api/solicitacoes (justificativa obrigatoria)
 - |
- + 2. Admin recebe solicitacao pendente
 - | +- APROVAR: POST /api/solicitacoes/<id>/aprovar
 - | | -> Status = DESLIGADO (para de ser pingado)
 - | +- REJEITAR: POST /api/solicitacoes/<id>/rejeitar
 - | | -> Motivo registrado, equipamento continua
 - |
- + 3. Religamento:
 - | +- Admin ou operador religa
 - | +- Status -> OFFLINE -> Ping -> ONLINE

14.3 Fluxo do Agente Remoto

AGENTE REMOTO (area local)

- |
- + 1. Inicia e busca config do servidor
 - | +- GET /api/agente/config (API key)
- + 2. Loop principal:
 - | +- GET /api/agente/equipamentos
 - | +- Pinga todos (rede local, latencia menor)
 - | +- POST /api/agente/heartbeat + ping-results
 - | +- Se servidor OFFLINE: salva em SQLite local
 - |
- + 3. Quando servidor volta:
 - | +- POST /api/agente/ping-results-batch (buffer)
 - | +- POST /api/agente/snapshots-diarios-batch
 - | +- POST /api/agente/snapshots-mensais-batch
 - |
- + 4. Se agente cai (> 180s sem heartbeat):
 - | +- PingService ativa FALLBACK (servidor assume)

14.4 Fluxo de Inicializacao do Sistema

```
python run.py
|
+- 1. argparse: --host, --port, --debug, --no-monitor, --no-bi
|
+- 2. Se --export-bi: exporta CSVs e sai
|
+- 3. Se --no-monitor:
|   +- create_app() (sem ping)
|   Senao:
|     +- create_app_with_ping()
|     +- ping_service.init_app(app) + start()
|
+- 4. Se BI ativo:
|   +- Thread daemon: run_bi_scheduler_thread()
|   +- Loop: rotina_20_minutos / rotina_meia_noite
|
+- 5. app.run(host, port, debug, threaded=True)
```

14.5 Categorias de Problema (Relatorios)

Categoria	Descricao
Rede	Problemas de conectividade, switch, cabo
Hardware	Defeito fisico no equipamento
Software	Bug, travamento, firmware desatualizado
Energia	Queda de energia, nobreak, fonte
Configuracao	Erro de configuracao de rede/IP
Infraestrutura	Problemas na infraestrutura do local
Outro	Outros problemas nao categorizados

Security Monitor v2.1

Hidroviás do Brasil
TI Segurança Patrimonial

Documentação gerada em 14/02/2026

Autor: Erik Thiago Barros Pinheiro

Documento confidencial - Uso interno Hidroviás do Brasil